

#Step 1 → Gathering Information Necessary

- Variables Needed
 - Grid with every row and column listed
 - players = [(black_stone, "Black"), (white_stone, "White")]
 - black_stone = chr(9675)
 - white_stone = chr(9679)

#Step 2 → Get user input

- From what the user inputs, determine which column and row they are referring to
- Also, determine if they want to stop the game or continue playing

#Step 3 → Build Program

- First, create the 9 x 9 grid using for loops
 - For every move made on the board, print the grid to display it
 - The loop should continue until the user enters "Stop"
- To place the stone, the program should check if the space is empty (has only a period) and if so a stone from whichever player can be placed
- Keep these rules in mind for running the program
 - If a user tries to place a stone where one already exists, print a message to the screen and try again.
 - To do this we must create a while loop that runs until the user places a stone in an area that isn't already filled
 - Alternate turns with black placing the first stone.
 - Create a loop that alternates between each key and value in a list to first match the black player to black stone and then white player to white stone
 - Other than that, you don't need to worry about placing stones on forbidden points, removing stones, determining a winner, etc

#Step 4 → Test Program

- Input: 0, 0 (Edge)
 - Output: (Top left corner should be filled in based off player)
- Input 9, 9 (Edge)
 - Output: (Bottom right corner should be filled in based off player)

- Input 1, 2 (Typical)
 - Output: (First row second column should be filled in based off player)
- Input 4, 3 (Typical)
 - Output: (Fourth row third column should be filled in based off player)
- Input 4, 3 (Typical)
 - Output: (Should give an invalidation message and allow user to try again based off previous inputs)
- Input 5, 6 (Typical)
 - Output: (Fifth row sixth column should be filled in based off player)